



POZNAN UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTING AND TELECOMMUNICATION
Institute of Computing Science

Master's thesis

ANALYSIS OF THE POST-QUANTUM DIGITAL SIGNATURE ALGORITHM SPHINCS+

inż. Filip Kokosza, 145210

Supervisor
dr inż. Anna Grocholewska-Czuryło

POZNAŃ 2024

Tutaj będzie karta pracy dyplomowej;
oryginał wstawiamy do wersji dla archiwum PP, w pozostałych kopiach wstawiamy ksero.

Contents

Abstract	1
Introduction	2
1 Literature review	3
2 Background on Digital Signatures	6
2.1 Introduction	6
2.2 Classical Digital Signature Algorithms	6
2.3 Possible Quantum Threats	7
3 Fundamentals of SPHINCS+	8
3.1 SPHINCS+ Structure	8
3.2 Lamport One-Time Signature	8
3.3 WOTS+: Winternitz One-Time Signature+	10
3.4 XMSS: eXtended Merkle Signature Scheme	14
3.5 FORS: Forest Of Random Subsets	17
4 Analysis of SPHINCS+	19
4.1 SPHINCS+ Hypertree	19
4.2 SPHINCS+ Parameters	20
4.3 Keys Generation	21
4.4 Signature Generation	22
4.5 Signature Verification	23
5 Security of SPHINCS+	26
5.1 Impact of the Parameters on the SPHINCS+ Security	26
5.2 Security Advantages of SPHINCS+	27
5.3 Security Disadvantages of SPHINCS+	27
6 Practical Implementations	28
6.1 SPHINCS+ Optimization	28
6.2 SPHINCS+ Practical Uses Cases	28
Conclusion	29
Bibliography	30

Abstract

TODO

Introduction

With the dynamic development of quantum computers and the growing threat from them, the necessity for quantum-resistant cryptography is rising. This issue also concerns digital signatures, which take a crucial role in ensuring the security of communication and data integrity. One of the safeguards against quantum mechanics is post-quantum algorithms. One of them that was created for digital signatures is SPHINCS+, which is also one of the finalists in the NIST standardization process of the post-quantum algorithms. Designed to be resistant to future threats from quantum computers, SPHINCS+ shows a well-proven security.

This thesis aims to analyze the post-quantum algorithm SPHINCS+ in terms of its structure, cryptographic mechanisms used, security, and practical implementations. Also, it will include an analysis of potential threats associated with using current standard digital signature algorithms.

The first chapter of this thesis sets the stage by presenting the literature review from the most recent papers gathered. It delves into different takes on quantum-resistant cryptography, and the main cryptographic mechanism and solutions used in SPHINCS+ are discussed. Also, this review provides a great understanding of the current state of post-quantum cryptography and the potential weaknesses that come with some of the algorithms.

The second chapter details the background of digital signature algorithms currently widely used in the industry. Basic concepts and principles that create classical cryptography are briefly discussed. Also, the potential threats posed by quantum computers in light of Shor's and Groove's algorithms are discussed.

In the third chapter, the focus shifts toward specific cryptographic mechanisms of the SPHINCS+ algorithm. The structure and fundamental cryptographic mechanisms that make this algorithm are explained in detail. The chapter covers algorithms, including WOTS+ (Winternitz One-Time Signature+), XMSS (eXtended Merkle Signature Scheme), and FORS (Forest Of Random Subsets) that are the basis of the SPHINCS+ design.

The fourth chapter is dedicated to the analysis of key aspects of the SPHINCS+ algorithm such as the SPHINCS+ Hypertree, keys generation, signature generation, and signature verification. The chapter presents a comprehensive explanation of every part of the SPHINCS+ algorithm.

The fifth chapter evaluates the advantages of the SPHINCS+ algorithm in the security context. The chapter discusses the impact of SPHINCS+ parameters on security and the sizes of keys, signatures. Also, potential weaknesses from the algorithm perspective are pointed out.

The sixth chapter discusses the possibilities of practical implementation of the SPHINCS+. Aspects of such implementations are pointed out. For instance different researched implication of the algorithm, performance, and data size optimization.

Chapter 1

Literature review

The development of quantum computers contributes to the rising quantum threat posed by them. Post-quantum cryptography is a direct response to this problem. Each year, it gains more recognition in terms of quantum-safe cryptography. Post-quantum digital signature algorithms such as SPHINCS+ are undergoing intensive research. The literature review reveals that many of the crucial aspects of this algorithm have been previously researched. However, some gaps in the study require further research and clarification.

SPHINCS+ algorithm is an expansion of the prior hash-based signature schemes, in particular WOTS (Winternitz One-Time Signature) and XMSS (eXtended Merkle Signature Scheme). The [13] article details the main structure and advantages that come with SPHINCS+, such as its statelessness and high quantum safety. SPHINCS+ introduced major improvements since its predecessor SPHINCS for instance, WOTS+ and FORS (Forest of Random Subsets), which are crucial in the leverage of the algorithm security. This article comes with a solid theoretical foundation, but because it is heavily technical, it could limit its accessibility to a wider audience.

Similar aspects are tackled in [3] paper, which also focuses on the structure and principles of SPHINCS+. The authors highlight the flexibility of the algorithm, allowing the security and performance to be adjusted to different needs. Yet, both articles would benefit from a more straightforward explanation of the mechanisms to make them more understandable to technicians outside the narrow cryptographic specialization.

The WOTS+ and XMSS signature schemes make the foundation for the SPHINCS+ algorithm. The WOTS+, compared to other hash-based signature schemes, provides lower memory and computing resource usage. Also, signatures generated by this scheme are shorter than in the previous WOTS, yet the security of the WOTS+ algorithm is maintained at a high level despite shorter signatures [12].

XMSS, just like WOTS+, is a main component in the design of SPHINCS+. As stated in [10], XMSS is a more advanced scheme based on Merkle trees. This paper delivers a detailed theoretical analysis of XMSS but lacks examples of its application in real systems. The [16] article presents the application of the XMSS signature scheme combined with Dilithium in the creation of a chain of trust. The authors propose integration with the TLS protocol which as proven comes with high quantum resistance.

Security issues related to SPHINCS+ and its components are key topics in the literature. Fault attacks pose great dangers, in which threat actor introduces errors into the computational process to gain access to secret information [9]. Authors of [9] propose various techniques that protect against such attacks. It is emphasized that safeguards at the algorithm design level can gain better security.

Similarly, the WOTS can be secured against quantum attacks in which the adversary has quantum access [15]. The authors presented an analysis that points out modifications that in turn, can boost the quantum resilience of WOTS. Another important aspect is the formal security verification of the XMSS addressed in the article [2]. The authors use formal proof-of-concept methods to ensure XMSS, therefore SPHINCS+ resistance to various types of attacks. Though this work is very technical, it presents compelling security evidence related to those algorithms.

Also worth mentioning are articles [14] and [11], which analyzes in detail and confirms the tightness of the security proof for the SPHINCS+ algorithm and other post-quantum digital signature algorithms. Strict security requirements are met by SPHINCS+, which authors proved by performing mathematical proofs. This work is crucial for understanding what the security foundations of this algorithm are and where the potential weaknesses may lie.

Data optimization and compression are the main issues of SPHINCS+, especially in applications with lesser computational capabilities. That is why the signature size and computing speed are important in those instances. An innovative approach to compression was introduced in the [14] article. The authors proposed an experimental method of reducing the size of signatures without significantly affecting the performance or security levels. However, there is no detailed analysis of the compatibility of these techniques with existing cryptographic systems. Similarly, a comparison was conducted of various post-quantum algorithms to the SPHINCS+ by [17]. Tests were carried out under various conditions, analyzing signature generation, verification time, and data size. The results of those tests are neatly presented in the paper. Also, the authors pointed out potential areas of optimization.

Implementing post-quantum cryptographic algorithms is a widely researched topic. Many researchers opted for dedicated hardware architectures for the algorithms, such as SPHINCS+, to improve their performance speed [8]. The authors discussed diverse hardware design approaches that could efficiently support the execution of post-quantum cryptographic algorithms and presented test results that showed how the hardware optimization accelerates the performance of these algorithms.

As mentioned before, increasing the performance of SPHINCS+ is one of the main goals for the practical implementation of this and any other post-quantum algorithm. That was explored by [1], where authors analyzed how parallelism can affect the signature generation time of SPHINCS+ on multi-core systems.

Several articles provide a broad overview of the current research state on post-quantum cryptography, examining multiple approaches and schemes [4],[7]. Both articles offer an overview of current research in the field, discussing hash-based algorithms as well as those using lattice structures and other techniques. These works are a solid theoretical foundation for comprehension of currently developed post-quantum algorithms. Also, given that the SPHINCS+ algorithm is one of NIST finalist for the standardization of post-quantum algorithms it undergoes many further developments and research [6].

The article [18] covers an overview of hash-based post-quantum algorithms that discusses their evolution, advantages, and disadvantages. This paper is vital for understanding the historical and theoretical context of hash-based signatures as well as their role in post-quantum cryptography. Many schemes are described in the paper, for instance, the Lamport OTS, HORS, WOTS+, XMSS, SPHINCS+, and many variations of SPHINCS+ algorithms.

One recent interesting topic is the Quantum Merkle Tree. Though it is not necessarily connected to post-quantum cryptography, it still provides valuable information on potential innovations of the classical Merkle tree in the quantum environment [5].

The literature on SPHINCS+ and other post-quantum cryptography is rich and gives valuable theoretical information. However, there is a need for further research, especially in the practical implementation, optimization, and security. Most existing work mainly focuses on theoretical aspects, often in a difficult way to understand for those outside the post-quantum specialization. This paper will focus on simplifying these concepts and offer more understandable and practical explanations that can potentially support the implementation of these advanced algorithms in real-life scenarios.

Chapter 2

Background on Digital Signatures

2.1 Introduction

Digital signatures are one of the most important tools used in current cryptography that ensure integrity, authenticity, and non-repudiation of any signed document or data sent digitally. In practice, a digital signature has a similar role to a handwritten signature but relies on cryptographic techniques to be created.

Commonly, a digital signature is created by first generating the fingerprint of the to-be signed document and then a private key of the signer is used to compute a signature. This guarantees that the signed document comes from a specific sender and its content has not been altered during transit. This implicates several security features:

- Integrity: Changes made to the document can be detected.
- Authenticity: A private key used to sign document verifies sender's identity.
- Non-repudiation: A sender cannot deny signing a message, their identity is implicated by the signature.

The verification of the signed document is fairly simple. A public key of the sender is used to compute the original fingerprint of the document, then a new hash of the document is generated. When the two digests are identical, it implies that the document's content has not changed.

Digital signatures are used in Internet banking to secure transactions and verify customers' identities; in public key infrastructure (PKI), they are used to issue digital certificates to secure Internet connections and communications. Digital signatures are also equivalent to handwritten signatures, which enable the signing of legal documents through a web browser.

2.2 Classical Digital Signature Algorithms

The current standard of digital signature algorithms consists of three algorithms: RSA, **EcDSA**, and ECDSA. To generate a digital signature, every algorithm takes as an input a hash value of the file, for example, generated by SHA3-256 from the SHA-3 family. Their security is based on the difficulty of solving complex mathematical problems in polynomial time. As for 2024 the previous algorithm DSA (Digital Signature Algorithm) was withdrawn from the standard due to security issues.

The RSA (Rivest-Shamir-Adleman) is one of the oldest yet the most popularly used in current cryptographic systems. Its security relies on the difficulty of factorizing large integers. The public key calculated by this algorithm is the product of two large prime numbers $N = pq$. While this

operation is easy to calculate, the decomposition of N into prime factors (p and q) is very time-consuming.

The ECDSA (Elliptic Curves Digital Signature Algorithm) algorithm is a variant of the earlier DSA. It incorporates the elliptic curve cryptography used to generate public and private keys. This algorithm depends on the complexity of solving the elliptic curve discrete logarithm problem to calculate the private key value.

On the other hand, the EdDSA (Edwards-Curve Digital Signature Algorithm) is one of the newest algorithms in the digital signature standard, in 2023 replacing the DSA algorithm. Compared to ECDSA, it provides enhanced security with deterministic derivation of nonces which removes reliance on the randomness present in ECDSA. Though it is based on the same elliptic curve discrete logarithm problem as the ECDSA, it utilizes more efficient Edwards curves, making the algorithm safer and easier to implement.

2.3 Possible Quantum Threats

In the age of large-scale quantum computers standard digital signature algorithms explained in section 2.2 may become susceptible or even broken by the quantum attacks. In 1994 Shor developed a quantum algorithm that poses a major threat to the standard algorithms. The algorithm is believed to solve the integer factorization problem and the discrete logarithm problem in polynomial time $O((\log N)^3)$. Both of these mathematical problems are the basis of multiple classical cryptographic algorithms, which create a valid threat to their security.

Another quantum algorithm that can be a threat to standard cryptography, is Grover's algorithm. This algorithm is designed to solve the unstructured search problem, where when given a set of N elements, it would require at most $O(N)$ searches to find the sought data of this set. Grover's algorithms shrink this time to only $O(\sqrt{N})$ tries, an improvement in search speed. Even though it doesn't affect any of the explained earlier digital signature algorithms, it for sure creates a security threat to other algorithms, such as SHA or AES because it decreases the strength of a key by half.

Considering those valid quantum threats in 2017, NIST initiated a standardization process of post-quantum algorithms that could withstand attacks from quantum computers. One of the finalists is the digital signature algorithm SPHINCS+, which is described in detail in the later chapters of this paper.

Chapter 3

Fundamentals of SPHINCS+

3.1 SPHINCS+ Structure

In hash-based cryptography, exist two main types of signature schemes, stateless and stateful. Both of them work similarly to achieve a high level of security, yet the main element that distinguishes them is how they approach the state of the secret keys.

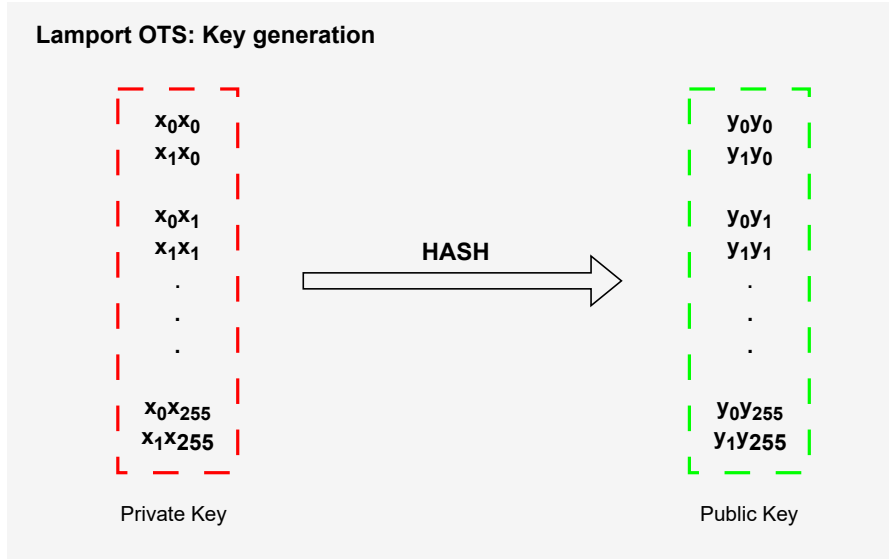
In stateful algorithms such as XMSS, one-time signatures are utilized, which means that there is a fixed number of signatures that can be derived from a single key. Keys used in such algorithms can only be used once, if used twice the attacker could forge their valid signatures. That is why in stateful signature schemes signer must keep track of keys, that have already been used requiring a lot of storage resources.

On the other hand, stateless signature schemes like FORS use few-time signatures. It implies that the signer does not have to remember every key used for a specific signature. The few-time signature schemes rely on large numbers of available keys chosen randomly, which makes the probability of choosing the same key twice very unlikely. The number of signatures per key is still limited because, in both schemes, a piece of secret information is released with every new signature, but the security decreases gradually.

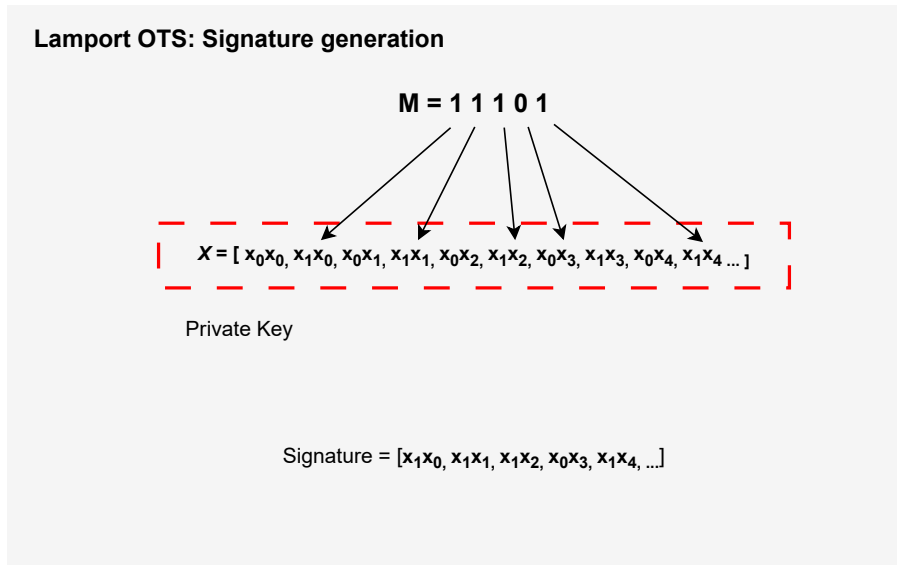
SPHINCS+ is a stateless hash-based algorithm composed of three main cryptographic mechanisms: WOTS+, XMSS, and FORS. All of them are part of SPHINCS+ Hypertree, details of how they function are described in 3.3, 3.4, and 3.5 sections.

3.2 Lamport One-Time Signature

One of the oldest signature schemes is the Lamport One-Time Signature created by Leslie Lamport. The key generation procedure is fairly simple, as shown in Figure 1 below. The examples shown later assume a SHA-256 is used for the signed message digest.

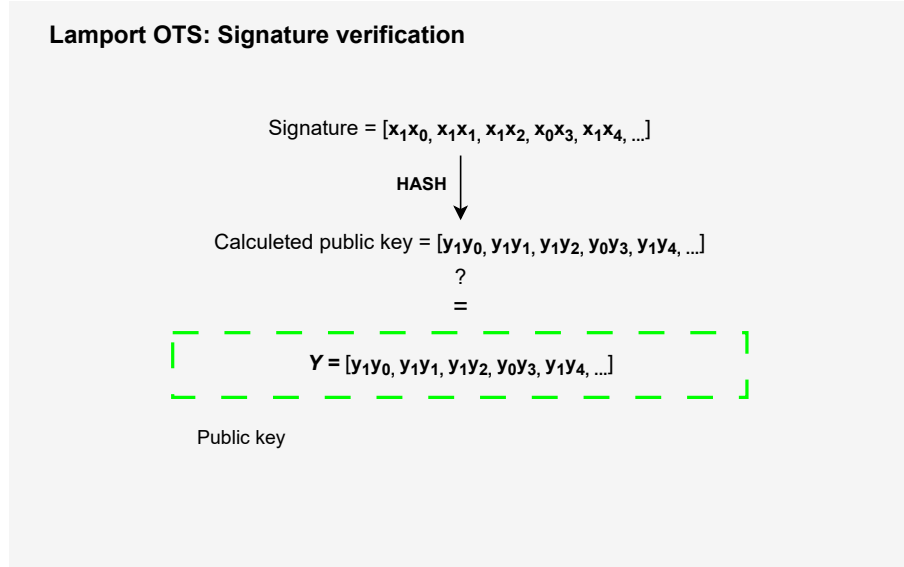
**Figure 3.1.** Lamport OTS: Key generation

As seen in Figure 3.1, the private key consists of 256 pairs for each 0 and 1 bit that can occur in the message, which in total gives a 512-bit private key. The public key is generated by simply, hashing the private key values, which creates a 512-bit public key.

**Figure 3.2.** Lamport OTS: Signature generation

The signature generation for the first 5 bits of the message digest M presented in Figure 3.2 goes as follows: iterating through the message if the first bit is 1 choose x_1x_0 if it is 0 take x_0x_0 private key, the algorithm iterates through the whole message giving the signature for this example as $\text{Signature} = [x_1x_0, x_1x_1, x_1x_2, x_0x_3, x_1x_4, \dots]$.

The receiver gets the message, signature, and the public key of the signer. This allows us to verify the signature, which is explained in Figure 3.3.

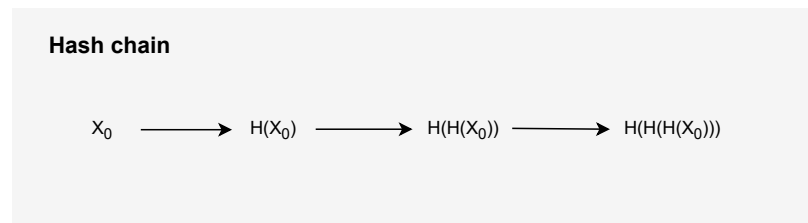
**Figure 3.3.** Lamport OTS: Signature verification

The verification process requires the receiver to hash each of the signature values. Then reconstruct a public key from the received public key set, selecting values based on the bits of message digest. After that, compare both of the public keys (the calculated, and the received one), when the values of the hashed signature match with the public key, the message is verified.

This simple algorithm's security heavily relies on the security of used one-way function typically a hash function. Because each time a message is signed, half of the secret key is revealed, so the same secret key cannot be used more than once. Also, the Lamport One-Time Signature scheme requires the transit of a big package to be sent to verify the signature. For the example above the total of bits sent to the receiver is 1048-bit, which is not very practical. The WOTS and later WOTS+ surpassed this algorithm, giving the ability to create shorter signatures.

3.3 WOTS+: Winternitz One-Time Signature+

The WOTS is a type of an OTS (One-Time Signature) same as Lamport OTS, but greatly differs from it. This signature scheme operates on the chain of hashes, which is constructed by hashing the value of the previous hash in the chain as shown in Figure 3.4. In fact, only the value of the starting element is needed to create a complete hash chain.

**Figure 3.4.** Example of hash chain

Recently, a new updated version of this signature scheme arose, called WOTS+. The main difference in the algorithm is adding a set of values generated with a pseudo-random function called PRF. Each value of this set is XORed with the corresponding value in the hash chain before the next hashing operation.

The algorithm uses 2 parameters that can be manipulated to adjust the time-memory trade-off.

- w - the Winternitz parameter
- m - the length of message, private key, and public key in bytes

The Winternitz parameter can take any value > 1 , but the recommended value can be chosen from the set $\{4, 16\}$, it gives the best time-memory trade-off and more practical implementations [20]. The m parameter usually takes the output length of the used one-way function, for example, if a SHA-256 hash function is used in the algorithm, the message length will be 32 bytes (256 bits), and the private and public keys. The effect of those parameters on the algorithm is discussed later in an example.

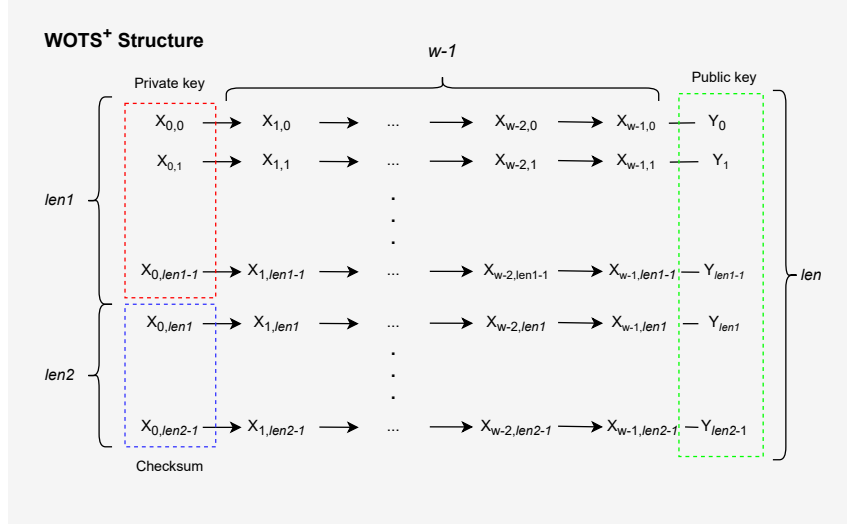


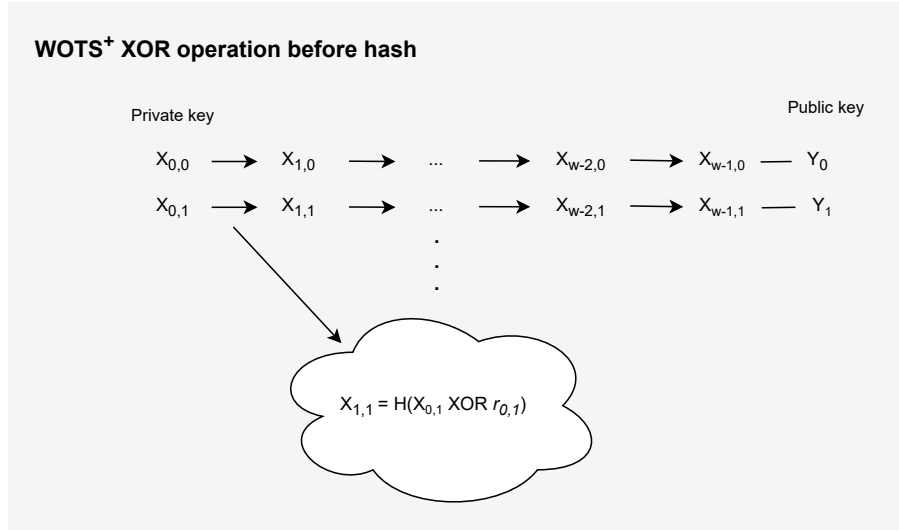
Figure 3.5. WOTS+ structure

Figure 3.5 shows the WOTS+ structure (each arrow indicate a hash operation), and how the w and m parameters affect it. The length of a hash chain (excluding the secret key element) is determined by the equation $w - 1$ while the number of hash chains, thus private keys, and public keys is set by the len parameter, which consists of $len1$ and $len2$. The len is calculated using the equation below.

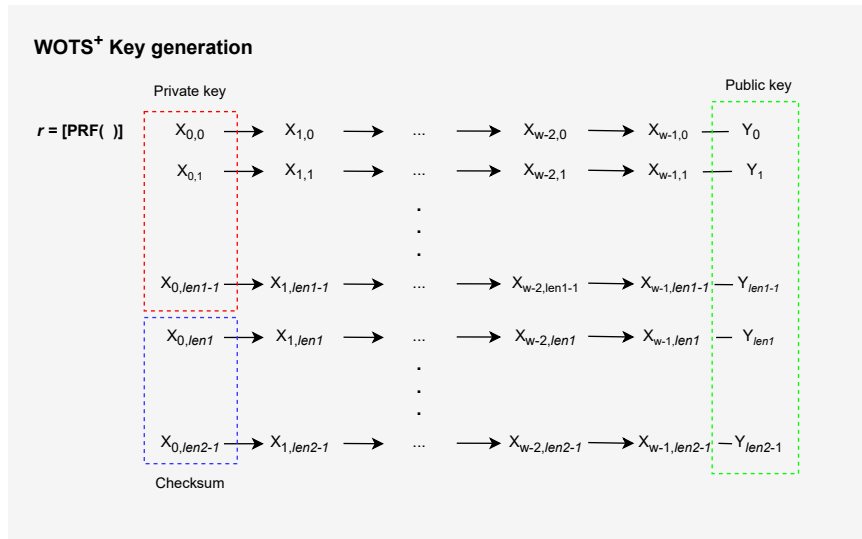
$$len_1 = \left\lceil \frac{8n}{\log w} \right\rceil, len_2 = \left\lceil \frac{\log(len_1(w-1))}{\log w} + 1 \right\rceil, len = len_1 + len_2$$

To prevent some of the attacks, WOTS+ uses a checksum that is added to the signature, verifying that the message has not changed. The number of chains regarding the message is described as the value of $len1$, while the number of checksum-related hash chains is represented by $len2$ as shown in the Figure 3.5.

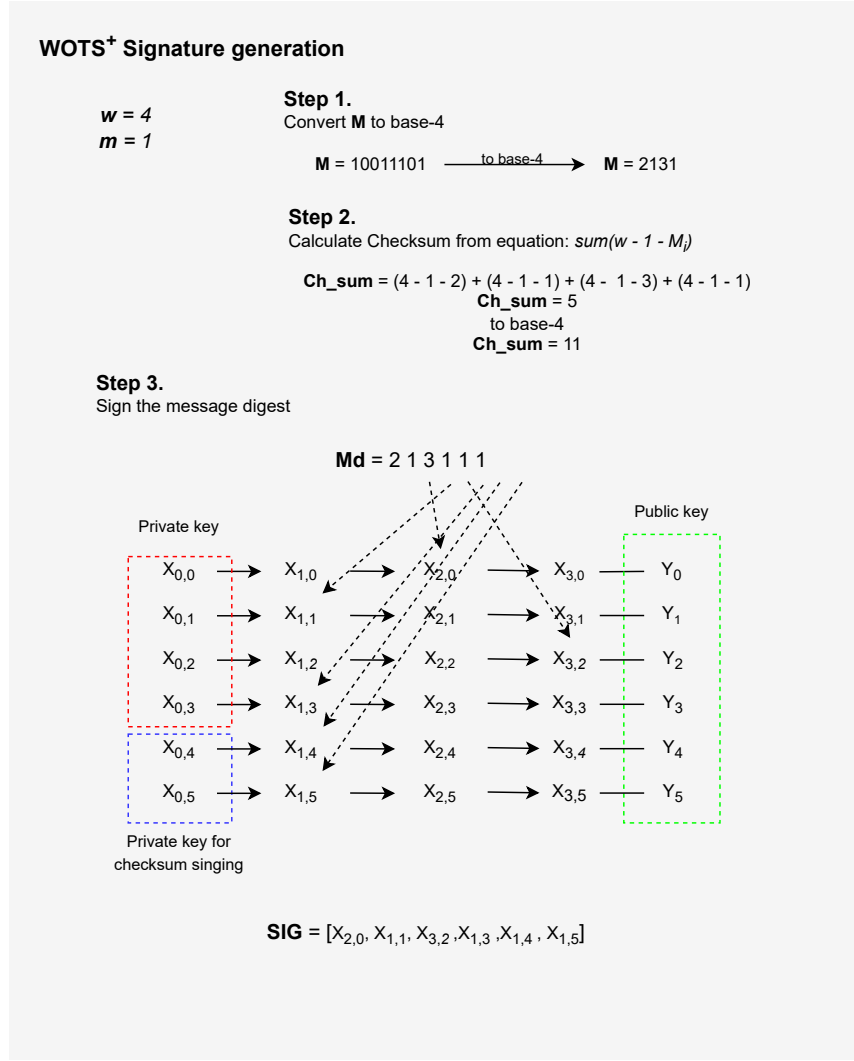
To create a chain of hashes WOTS+ uses additional steps compared to previous WOTS. Before hashing the prior value, a PRF generates a r the r two-dimensional array of len sets containing $w - 1$ pseudo-random m -byte values (also called bitmasks) for each step in the chain. Then a corresponding bitmask from the r array is applied to the element of the chain using the XOR operation, after that the output of this operation is hashed. This is shown in Figure 3.6 below.

Figure 3.6. WOTS⁺ hash chain operations

The key generation process begins by generating a $len1$ -element array of m -byte private keys, using a PRF. In the next step, the r array is generated as explained earlier. Then a chain of hashes is calculated using $w - 1$ times the algorithm shown in Figure 3.6. After the iterations, the public key is generated, which is the last element of the hash chain.

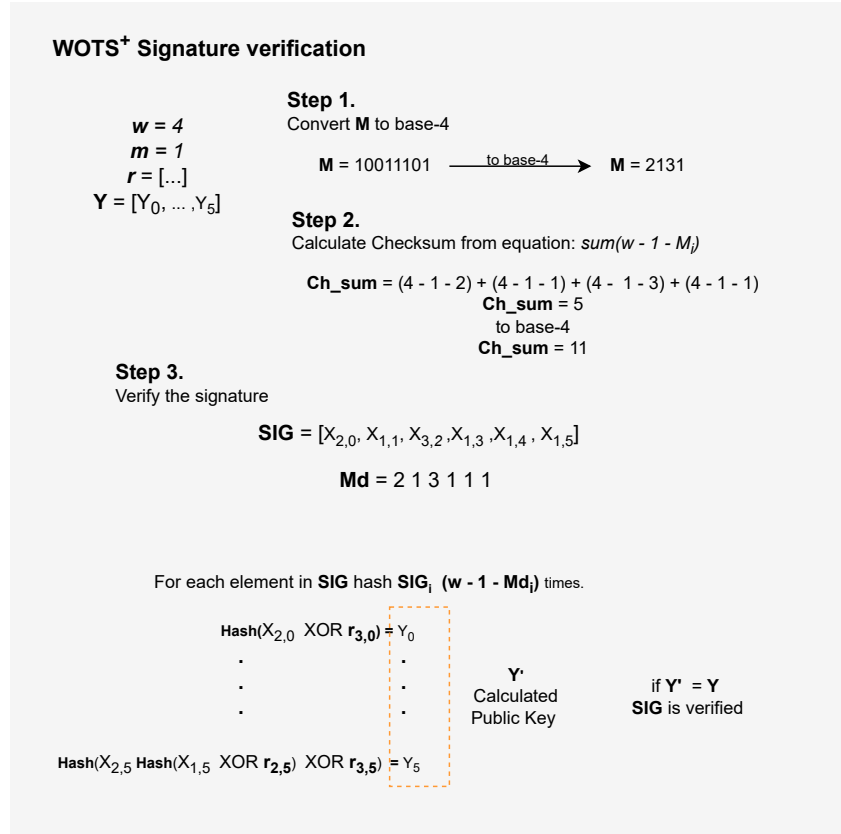
Figure 3.7. WOTS⁺ Key generation

Signature generation includes some processing to the message digest. First, the message digest is converted from binary into base- w representation. Second, the checksum is calculated by summing up the differences between the maximum possible digit in base- w . After that, these values are concatenated together creating a ready-to-sign message. Each of the digits from this digest represents the position of a private key in the chain of hashes. Lastly, the elements pointed by the digest digits are collected and create the message signature. An example of signature generation is shown in the Figure 3.8.

Figure 3.8. WOTS⁺ Signature generation

Example in Figure 3.8 uses $w = 4$ and $m = 1$ -byte parameters. Based on those parameters key generation algorithm is run. The message digest is $M = 10011101$. The first step include transformation of the M from binary to base-4 representation. In the next step checksum is calculated, then it is converted to base-4. Md is created by concatenating M and Ch_sum base-4 values. Then algorithm iterates through each digit of the message digest, the digit points to specific hash of the hash chain. After that iteration a signature SIG is created.

The verification of the signature is performed by further hashing a set number of times to achieve the public key values. The receiver has a message, signature and the w , m and r array are known to them. The first step to verify the signature is to perform the same processing of the message as explained in the signature generation algorithm. After the digest is in the base- w and the checksum is calculated. The receiver must know, how many steps are required to reach the end of the hash chain (the public key). This is done by subtracting the digest digit from the $w - 1$. The first digit from the message corresponds with the first hash chain and so on. Finally, the receiver performs additional XOR and hash operations on the signature. If the further calculated signature has the same values as the public key, the message is verified.

Figure 3.9. WOTS⁺ Signature verification

To optimize this algorithm in practical implementation, the bitmasks and the chain of hashes are generated "on-the-fly". This lowers the size of data needed to be stored, which does not affect the speed of the computation significantly [12].

3.4 XMSS: eXtended Merkle Signature Scheme

The Merkle Tree is the foundation of XMSS, it is represented by a balanced binary tree of hashes. The computation process of such a tree consists of concatenating the values of two sibling nodes and then applying a one-way function to get the parent hash. The process repeats until the root node hash is calculated. Merkle Tree is used by many signature schemes to lower the size of the public key by placing the private keys at the leaves of the binary tree, where the root node is the public key.

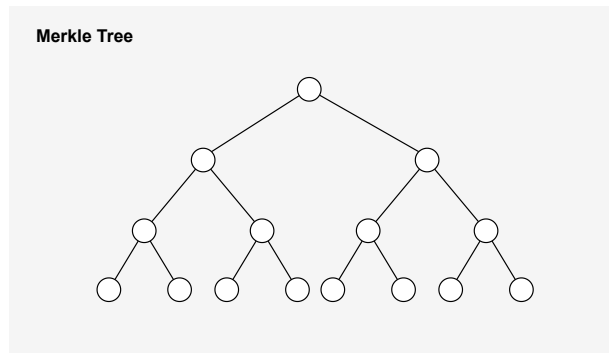


Figure 3.10. Merkle tree example

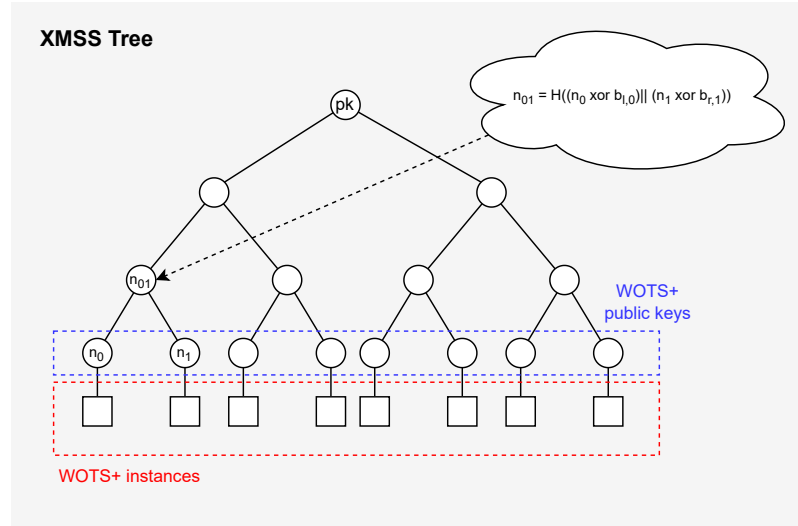


Figure 3.12. XMSS tree structure - node calculation method

To sign a message, the algorithm must choose a WOTS+ private key that was never used. After that, the message is signed by the WOTS+ algorithm as shown in 3.3. Then a path in the tree is chosen based on the used WOTS+ private keys. After that, a signature of such a message contains a WOTS+ signature and authentication path. If a tree is of height h , the path consists of h hashes.

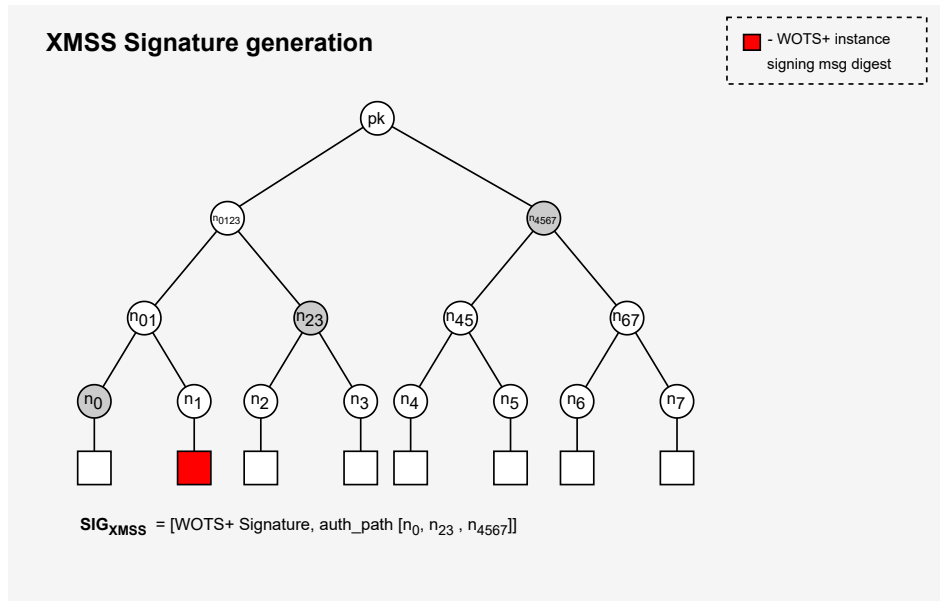


Figure 3.13. XMSS signature generation

The verification process is similar to the Merkle Tree signature scheme, but in the calculations of the parent nodes, a bitmask is in the same fashion as during the key generation. First, the WOTS+ part of the signature is verified using the WOTS+ verification process 3.3, if the calculated public key equals the WOTS+ public key, then the verification continues. The next and last step is calculating the XMSS public key, by using the values provided in the authentication path. Both of these steps must be completed to successfully verify the signature.

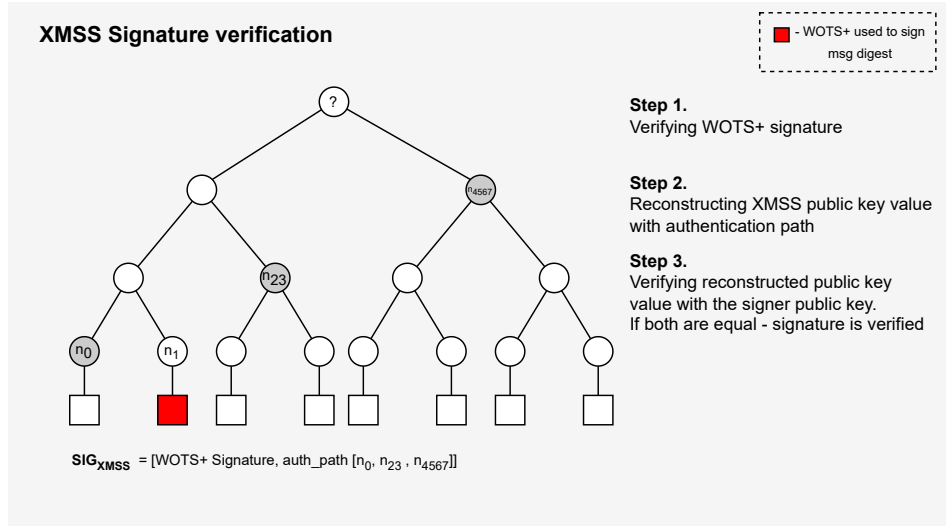


Figure 3.14. XMSS signature generation

In the SPHINCS+, the XMSS construction is used in the form of an $XMSS^{MT}$, a multi-tree version of detailed XMSS. The structure of such a tree is described in 4 section.

3.5 FORS: Forest Of Random Subsets

FORS is a fairly new signature scheme, introduced in the SPHINCS+ signature scheme. FORS is a few-time signature scheme, because of that it is a stateless hash-based signature scheme [17]. The private keys of FORS can be used more than once to sign a message, it is due to the number of private keys that are generated when FORS trees are first generated.

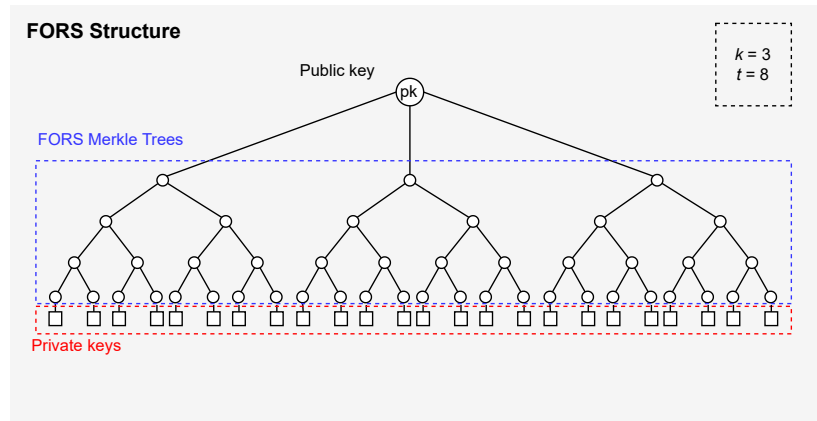


Figure 3.15. FORS Structure

The FORS structure is shown in Figure 3.15, specific parameters are used to build such a tree, which determine its height, number of private keys, and number of trees.

- k - the number of trees to be generated
- t - the number of private keys for a single tree (an equation is used to calculate this parameter $t = 2^a$), also the max length of the message that can be signed
- a - the height of a single tree

Considering that each tree consists of t private keys, the whole number of private keys generated by this scheme is kt . The secret keys are randomly generated by a chosen PRF, and then to calculate the FORS public key it requires every value from the root nodes of the trees. To obtain those, the algorithm follows a similar approach as in the Merkle Tree 3.4. Internal nodes are constructed by hashing pairs of child nodes together, up until each tree root is calculated. Finally, all k root node values are concatenated and a one-way function is applied to create a public key.

To sign a message, the signer must divide the binary message digest into k a -bit chunks. Each k -th chunk is then signed using the private keys of the k -th tree. The chosen private keys for the signature are dictated by the integer value of the chunk (see Figure 3.16 below). The signature again consists of chosen leaf nodes, and respective authentication paths, highlighted with red color and grey color in the Figure 3.16 respectively.

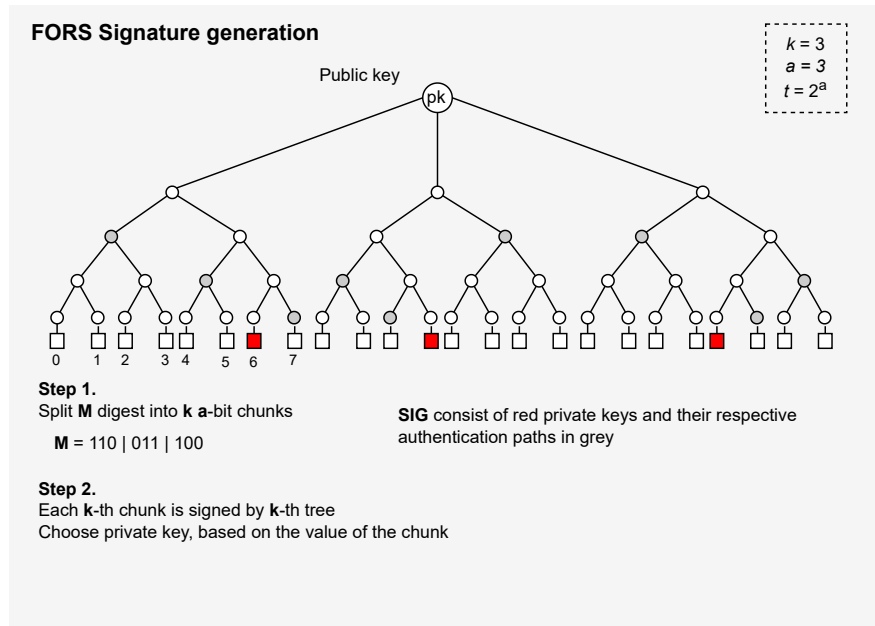


Figure 3.16. FORS Signature generation

To verify the message, the receiver must reconstruct each k root nodes. This is done the same way as in Figure 3.16, k authentication paths for the respective k private keys is used to calculate each of the k root nodes. Then a FORS public key is calculated from those values, if both the signer public key and the freshly computed public key from the signature match, the message is verified.

The important piece of information to note is that in SPHINCS+ the FORS signature is not explicitly verified [3]. Instead, it is verified with the WOTS+ signature, which is described in detail in 4 chapter.

Chapter 4

Analysis of SPHINCS+

4.1 SPHINCS+ Hypertree

SPHINCS+ uses a specific structure to sign and verify the messages, called SPHINCS+ Hypertree. This tree-of-trees incorporates every signature scheme explained in chapter 3 WOTS+, XMSS (in the Multi-Tree variant), and FORS. An example of such a Hypertree is seen in the Figure 4.1.

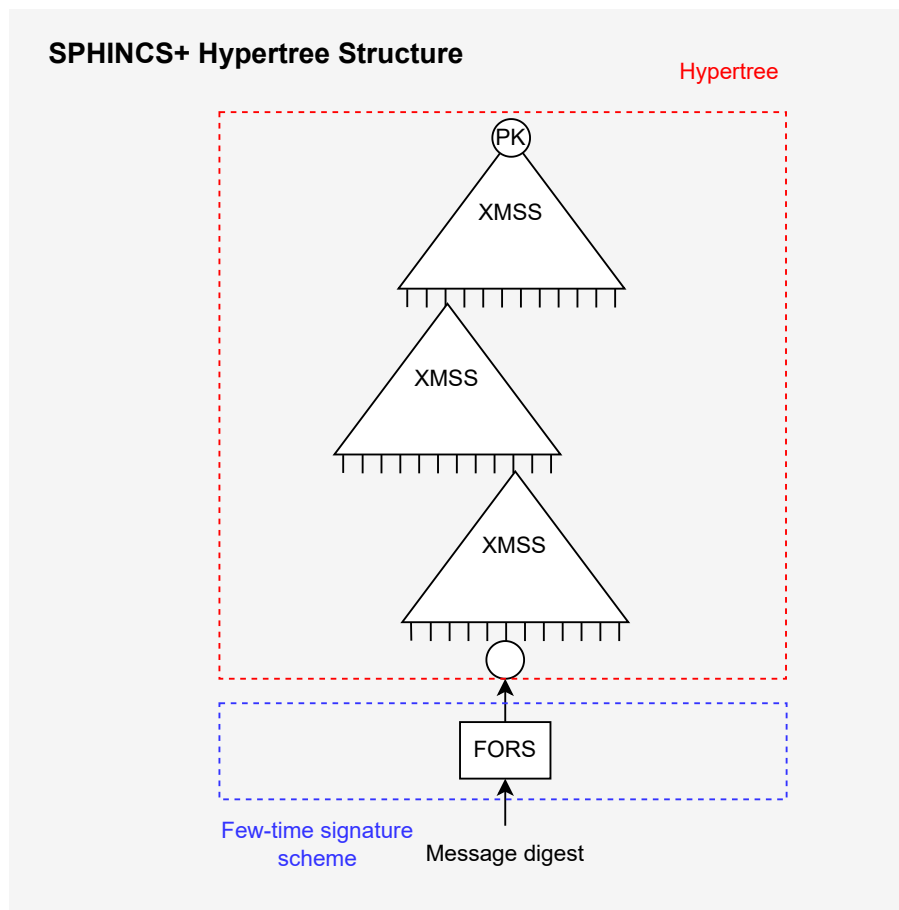


Figure 4.1. SPHINCS+ Hypertree structure

The Hypertree consists of many XMSS tree layers, and each of the trees is of the same height. At the bottom of the HT (Hypertree) is the few-time signature scheme FORS, whose private keys sign the message digest. The leaves of the bottom HT trees are the public keys of FORS trees.

Leaves of every tree in HT, are WOTS+ public key values, which sign either the FORS public key (the last tree of HT) or the root node of the XMSS tree. The top-most tree root is the SPHINCS+ public key.

The Hypertree structure is used to sign FORS public keys. This is done in the same way as for the single XMSS tree explained in 3.4. The main difference is that for the bottom tree, the WOTS+ does not sign the message digest, instead it signs the public key of the FORS instance. On the other HT layers, it behaves like XMSS^{MT}, so the private keys of each tree (except the bottom one) are the public keys of the WOTS+ scheme, which during the signature phase, signs the lower XMSS tree root node.

This construction acts as a certification tree, where the signature is a path for certifying the FORS public key [3].

The HT signature contains multiple XMSS signatures (WOTS+ signatures, along with XMSS tree paths) and the FORS signature (private keys and authentication paths).

4.2 SPHINCS+ Parameters

Given the complexity of the SPHINCS+ Hypertree, it requires a couple of parameters for each of the mechanisms and schemes used in the algorithm. The SPHINCS+ parameters present as follows [13]:

- n - security parameter, decides private and public keys and signature length
- h - the height of the Hypertree
- d - the number of layers in the Hypertree (the layers excludes the FORS scheme tree)
- k - the number of trees in the FORS scheme, detailed in 3.5
- t - the number of leaves in FORS, detailed in 3.5
- w - the Winternitz parameter, used in all of the WOTS+ schemes, detailed in 3.3

The parameter len used in the WOTS+ scheme as the total number of hash chains is calculated the same as described in the 3.3 section.

The message digest length m for SPHINCS+ is calculated by the following equation:

$$\lfloor k \log t + 7/8 \rfloor + \lfloor h - h/d + 7/8 \rfloor + \lfloor h/d + 7/8 \rfloor$$

To make the SPHINCS+ easier to understand the following values of parameters are used in 4.3, 4.4 and 4.5 sections:

- $m = 32 - bit$ (message digest length calculated by above)
- $n = 8$ (using simple one-way function $H(x) = (x31) \bmod 256$)
- $h = 6$
- $d = 2$
- $k = 3$
- $t = 8$
- $w = 4$

These values are not the most optimal for SPHINCS+. The optimized parameter set for practical implementations is described in 5 chapter.

Described steps of key generation and signature generation/verification, omit the use of an implementation-optimized version of SPHINCS+, which makes many computations on-the-fly. These optimizations are discussed in the 6 chapter, sections 4.3, 4.4 and 4.5 purely focus on understanding the main idea behind the SPHINCS+ algorithm.

4.3 Keys Generation

For illustrative purposes, it is assumed that the Hypertree is generated fully during the key generation phase and stored.

The SPHINCS+ private keys are the FORS private keys that directly sign message bits. It is created using a pseudo-random function - PRF. During this initial state, WOTS+ private keys and later used bitmasks (within the WOTS+ and XMSS structure) are also generated by PRF.

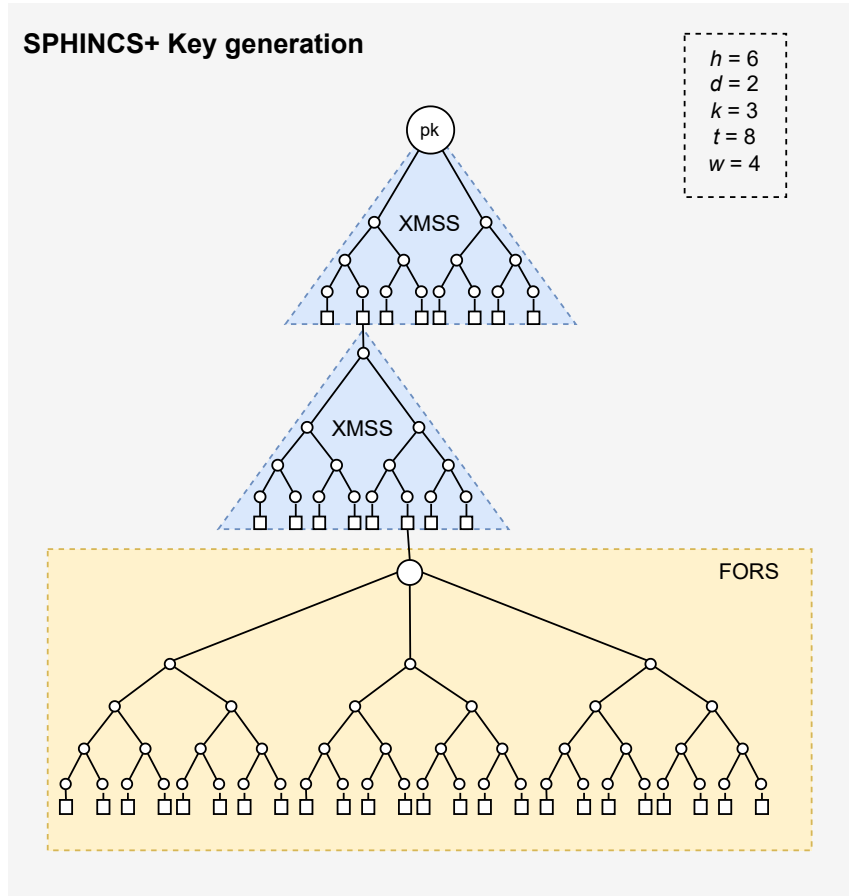


Figure 4.2. SPHINCS+ Key generation

SPHINCS+ public key is the root node of the top tree in the Hypertree structure. To obtain the value of this tree, entire HT tree nodes must be generated. This is done by calculating every FORS tree-of-trees public key (the leaves of the bottom XMSS tree), then WOTS+ signs the FORS public keys. The reconstruction of the HT on each level behaves as explained in 3.4, with the difference that for each XMSS tree the WOTS+ signs the root node of the tree below it.

4.4 Signature Generation

The signature generation algorithm is a composite of the previously discussed methods included in the SPHINCS+ Hypertree, with additional message digest transformations. SPHINCS+ signature includes the FORS signature (k private keys with the respective k authentication paths) and the HT signature (d WOTS+ signatures with respective d XMSS private keys and d authentication paths). The signing process involves choosing a path through the HT.

The SPHINCS+ signs only a specific part of the message digest msg_d , where the rest of this digest is used as an index to select the XMSS tree from the bottom HT layer tree_index and the FORS instance from the leaf of this XMSS tree leaf_index . The calculations are shown below.

- tmp_msg_d = first $\lfloor (ka + 7)/8 \rfloor$ bytes of digest
- tmp_tree_index = next $\lfloor (h - h/d + 7)/8 \rfloor$ bytes of digest
- tmp_leaf_index = next $\lfloor (h/d + 7)/8 \rfloor$ bytes of digest

Then another round of calculations is done to get the values of desired parameters:

- msg_d = first ka bits of tmp_msg_d
- tree_index = first $h - h/d$ bits of tmp_tree_index
- leaf_index = first h/d bits of tmp_leaf_index

In this instance, a 7-th FORS instance of the 5-th XMSS bottom tree is used to sign the 9 bits of the message digest. The FORS signature is generated as explained in 3.5. So, at this stage, the signature is shown in 4.3.

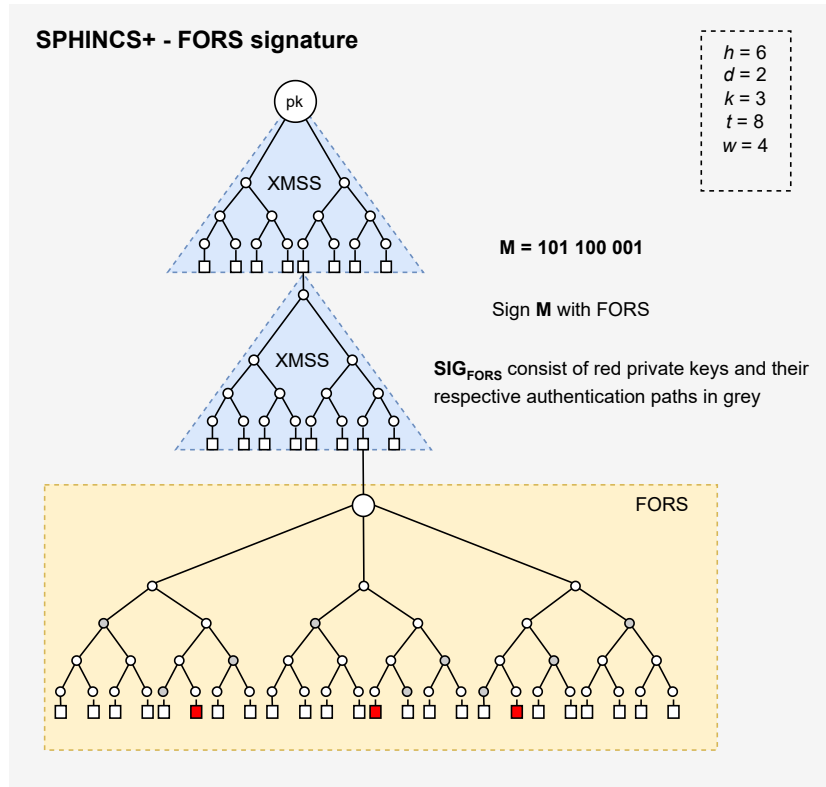


Figure 4.3. SPHINCS+ FORS signature of the digest

After that FORS public key is signed by the HT as described in 4.1. Finally, a signature is created from the FORS signature and the HT signature shown in the Figure 4.4.

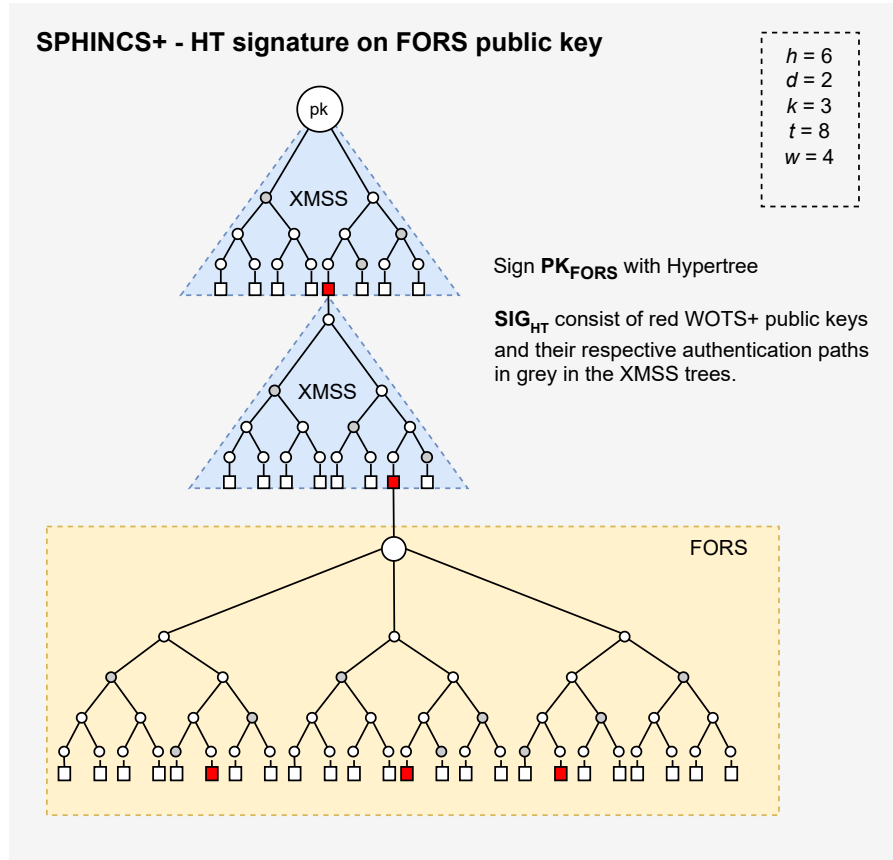


Figure 4.4. SPHINCS+ HT signature of the FORS public key

The grey nodes indicate the authentication path of FORS and XMSS, red nodes indicate chosen private keys, as well as WOTS+ public key nodes.

4.5 Signature Verification

To verify the SPHINCS+ signature, the receiver has to go through a similar process as the signer went signing the message. First, the message digest md and $tree_index$, $leaf_index$ indexes must be reconstructed using the same equations as in the signing process 4.4.

- $msg.d = 101100001$
- $tree_index = 110$
- $leaf_index = 100$

Then a FORS public key must be calculated using the FORS authentication path in the SPHINCS+ signature. This public key is verified at the HT level, not directly by comparing two FORS public key values (received, and calculated) [3].

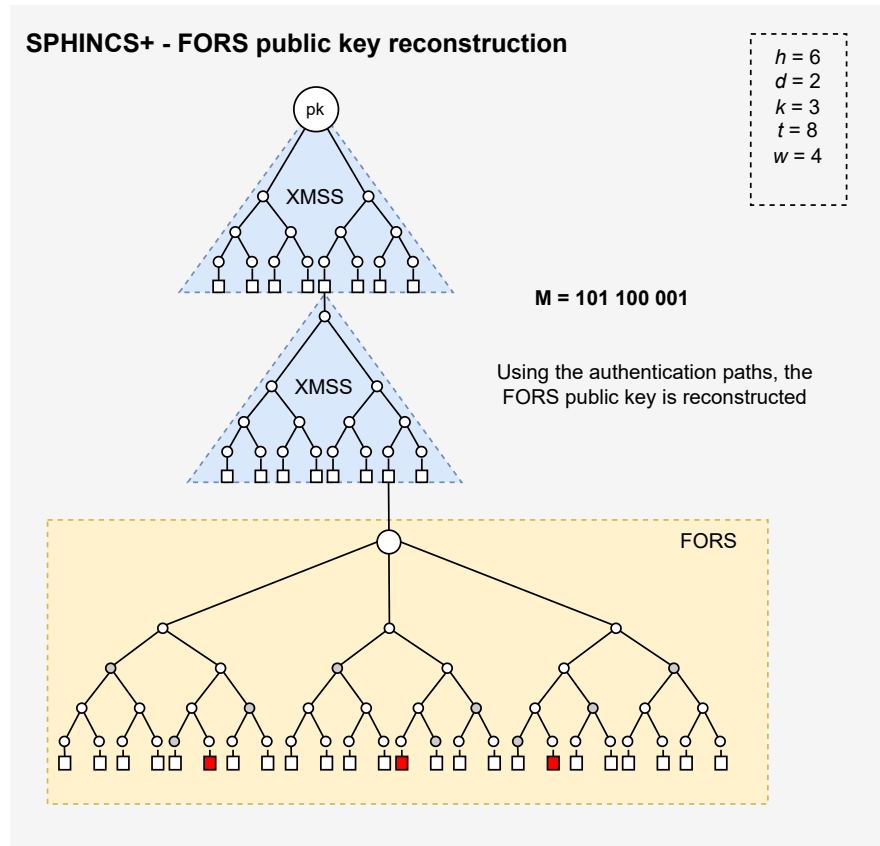


Figure 4.5. SPHINCS+ FORS public key reconstruction

The next step is to verify the Hypertree signature on that FORS public key. This is done by reconstructing each of the XMSS tree's root nodes with the received authentication path, which is described in 3.4 section.

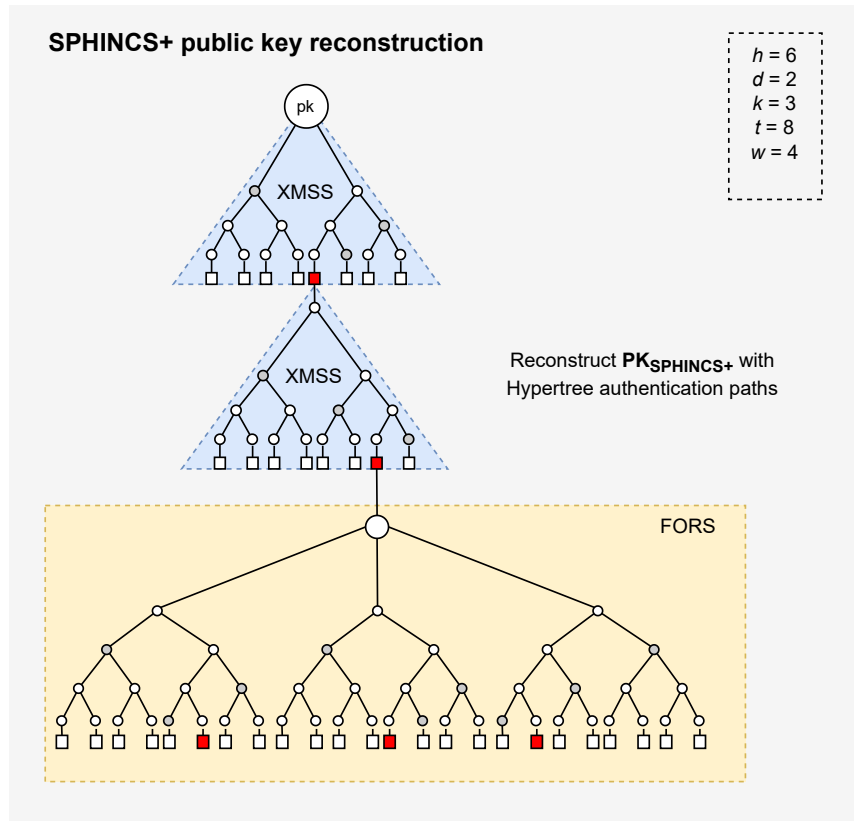


Figure 4.6. SPHINCS+ FORS public key reconstruction

If the computed SPHINCS+ public key equals the sender's public key, the message is verified. When the FORS public key does not match the real one, the HT verification will fail.

Chapter 5

Security of SPHINCS+

5.1 Impact of the Parameters on the SPHINCS+ Security

The selection of parameters in SPHINCS+ is crucial to the security and performance of the algorithm. Proper setting of such parameters can significantly affect the algorithm's resistance to attacks, including quantum attacks. Below the impact on the security of specific parameters is analyzed:

- n - the security parameter of SPHINCS+ directly affects the security level that a SPHINCS+ instance is able to reach. Larger values of n give a better security level, but it affects the size of each node in the Hypertree, thus making the final signature much larger.
- h - the height of the hypertree affects the number of FORS instances and the general size of the signature. Consequently, it decides the likelihood that a FORS key pair is used multiple times: Taller hypertrees have a much smaller probability of reusing a key pair, thus leading to higher security.
- d - the number of layers in the hypertree only affects the performance of the algorithm thereby larger d value makes the subtrees smaller so the signing is much faster.
- k - the number of subtrees of FORS combined with the t parameter affects the performance and the security of FORS.
- t - the number of leaves of the FORS trees must be a power of two. When the t is small, the signature size and computation speed decrease.
- w - the Winternitz parameter, controls the number and the length of hash chains in the WOTS+ instances. Greater w makes the signatures shorter, but the signing process takes much more time. Thus, it affects the SPHINCS+ size and speed, because most of the time the algorithm spends computing the WOTS+ chain of hashes.

Authors of SPHINCS+ created a set of parameters to achieve certain security levels established from 1 to 5 [13].

SPHINCS+ variant	n	h	d	$\log(t)$	k	w	security level	signature size (bytes)
SPHINCS+-128	16	63	7	12	14	16	1	7 856
SPHINCS+-192	24	63	7	14	17	16	3	16 224

SPHINCS ⁺ -256	32	64	8	14	22	16	5	28 792
---------------------------	----	----	---	----	----	----	---	--------

TABLE 5.1: Parameters sets for specific SPHINCS+ instances, based on data from the [13] documentation.

5.2 Security Advantages of SPHINCS+

SPHINCS+ is a quantum-resistant algorithm, relying on the second-preimage resistance property of the used hash function for its security. Hash functions are known to resist quantum attacks that use Shor’s algorithm. While classic algorithms like RSA can be broken by Shor’s, the hash cryptography used in SPHINCS+ remains secure. SPHINCS+’s cryptographic robustness against quantum algorithms such as Shor’s and Grover’s algorithm has been thoroughly discussed in the literature, confirming its superiority over other classical algorithms [4], [7].

The main strength of SPHINCS+ is its statelessness, meaning that the algorithm does not require to store the states of used private keys as explained in 3. Other stateful hash-based algorithms can be susceptible to many forgery attacks through improper state management this is not present in the SPHINCS+ due to the stateless design [3].

SPHINCS+ is designed to withstand side-channel attacks such as timing attacks, power analysis, and differential analysis. With the Hypertree structure, SPHINCS+ minimizes the risk of information leakage through side channels. In addition, the ability to generate signatures in SPHINCS+ non-deterministically by adding additional randomness (described in 6) makes it difficult for attackers to perform effective analysis [9], [2], [3].

SPHINCS+ offers great flexibility in the selection of parameters, such as HT height, signature, and private and public key length. This allows the algorithm to be adjusted to desired security levels to find the optimal signature size and computational cost of signing trade-off [14] (described in 5.1 section).

5.3 Security Disadvantages of SPHINCS+

One of the main drawbacks of SPHINCS+ is its relatively long signature length compared to classical digital signature algorithms. Although the longer signature length is due to the need for a higher level of security, this can be a problem in the context of memory and bandwidth constraints [14], [17].

The signature generation in SPHINCS+ requires way more resources than the traditional algorithms [17]. Even though the Hypertree structure and data optimization techniques help optimize the performance [13], SPHINCS+ still requires larger computing power. This limits its use in IoT (Internet of Things) devices that are resource-constrained.

The SPHINCS+ structure, consisting of multiple layers of Merkle trees creating the Hypertree [3] [10], is more complex than the structures used in other digital signature algorithms. This complexity can lead to potential implementation errors, which can affect the final security of the algorithm.

Chapter 6

Practical Implementations

6.1 SPHINCS+ Optimization

As mentioned in chapter 4, SPHINCS+ computes parts of the Hypertree on the fly, saving storage resources. This is done by introducing two seeds for the pseudo-random function used to generate the SPHINCS+ structure. The *SK.seed* is used to deterministically generate private keys used in WOTS+ and FORS instances using the PRF. The *PK.seed* is also used by the PRF, but for all the public components whenever randomness is needed [13]. Both of those seeds are kept within the private key of SPHINCS+. These values make the signature verifiable, meaning that each time the signature is being verified the SPHINCS+ HT is generated on the fly using those seeds, which ensures that the SPHINCS+ public key value will remain the same as during message signing.

To ensure that this mechanism is not exploited by the replay attacks, SPHINCS+ relies on the FORS message-based randomness. This means that the private keys as well as the path that will sign the message are based on the specific bits of the message (4.4).

As mentioned in the chapter 5, the SPHINCS+ parameters set is a great ground for further optimization. Tuning the parameters can result in faster computation times and better security.

Methods, such as parallelism could be used to improve the performance time. Also constructing dedicated hardware for SPHINCS+ can improve its speed generation time [8].

6.2 SPHINCS+ Practical Uses Cases

One of the main challenges of implementing SPHINCS+ is its performance, especially on the limited computing resources devices. Authors of [1] paper, tested the SPHINCS+ implementation on the mobile device using *Snapdragon 865* CPU. The implementation parallelized the SPHINCS+ algorithm, which resulted in a 15-time increase in performance, compared to standard sequential implementation.

In IoT devices, digital signatures are fundamental to security, protecting against MiTM (Man-in-The-Middle) and other tampering attacks. SPHINCS+ could find use in such devices, protecting them from quantum attacks, the parallelized version of SPHINCS+ implementation can be a foundation for such practical implementation.

Practical implementation of SPHINCS+ can be quite challenging, considering its complex structure. Careful research must be conducted before implementing such an algorithm, to prevent the introduction of additional vulnerabilities.

Conclusion

TODO

Bibliography

- [1] Gustavo Banegas and Florian Caullery. Multi-armed SPHINCS⁺. Cryptology ePrint Archive, Paper 2023/636, 2023.
- [2] Manuel Barbosa, François Dupressoir, Benjamin Grégoire, Andreas Hülsing, Matthias Meijers, and Pierre-Yves Strub. Machine-checked security for XMSS as in RFC 8391 and SPHINCS⁺. Cryptology ePrint Archive, Paper 2023/408, 2023.
- [3] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS⁺ signature framework. Cryptology ePrint Archive, Paper 2019/1086, 2019.
- [4] Daniel J. Bernstein and Tanja Lange. Post-quantum cryptography. *Nature*, 2017.
- [5] Lijie Chen and Ramis Movassagh. Quantum Merkle Trees. *Quantum*, 8:1380, 2024.
- [6] Lily Chen, Stephen Jordan, Yi-Kai Liu, Dustin Moody, Rene Peralta, Ray Perlner, and Daniel Smith-Tone. Report on post-quantum cryptography, 2016.
- [7] Duc-Thuan Dam, Thai-Ha Tran, Van-Phuc Hoang, Cong-Kha Pham, and Trong-Thuc Hoang. A survey of post-quantum cryptography: Start of a new race. *Cryptography*, 7(3), 2023.
- [8] Mohammed Nabeel Najwa Aaraj Marc Manzano Ramesh Karri Deepraj Soni, Kanad Basu. *Hardware Architectures for Post-Quantum Digital Signature Schemes*. Springer Journal, 2021.
- [9] Aymeric Genêt. On protecting SPHINCS⁺ against fault attacks. Cryptology ePrint Archive, Paper 2023/042, 2023.
- [10] A. Huelsing, D. Butin, S. Gazdag, J. Rijneveld, and A. Mohaisen. Rfc 8391: Xmss: extended merkle signature scheme, 2018.
- [11] Andreas Hülsing. *Practical Forward Secure Signatures using Minimal Security Assumptions*. PhD thesis, Technische Universität Darmstadt, Darmstadt, August 2013.
- [12] Andreas Hülsing. W-ots+ - shorter signatures for hash-based signature schemes. *IACR Cryptol. ePrint Arch.*, 2017:965, 2013.
- [13] Ward Beullens Christoph Dobraunig Maria Eichlseder Scott Fluhrer Stefan-Lukas Gazdag Andreas Hülsing Panos Kampanakis Stefan Kölbl Tanja Lange Martin M. Lauridsen Florian Mendel Ruben Niederhagen Christian Rechberger Joost Rijneveld Peter Schwabe Bas Westerbaan. Jean-Philippe Aumasson, Daniel J. Bernstein. Sphincs+ – submission to the 3rd round of the nist post-quantum project. v3.1, 2022.
- [14] Mikhail Kudinov, Andreas Hülsing, Eyal Ronen, and Eylon Yogev. SPHINCS+c: Compressing SPHINCS⁺ with (almost) no cost. Cryptology ePrint Archive, Paper 2022/778, 2022.
- [15] Christian Majenz, Channele Matadah Manfouo, and Maris Ozols. Quantum-access security of the winternitz one-time signature scheme. Cryptology ePrint Archive, Paper 2021/387, 2021.
- [16] Soundes Marzougui and Jean-Pierre Seifert. Xmss-based chain of trust. *EPiC Series in Computing*, 2021.

- [17] Filip Opiłka, Marcin Niemiec, Maria Gagliardi, and Michail Alexandros Kourtis. Performance analysis of post-quantum cryptography algorithms for digital signature. *Applied Sciences*, 14(12), 2024.
- [18] Vikas Srivastava, Anubhab Baksi, and Sumit Kumar Debnath. An overview of hash based signatures. *Cryptology ePrint Archive*, Paper 2023/411, 2023.